Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

smelt-ml: A Pure-Rust Machine Learning Framework Combining Spatial Modeling, Conformal Prediction, and Cache-Optimized Gradient Boosting

Francisco Parra
Universidad de Santiago de Chile

Abstract

We present **smelt-ml**, an open-source machine learning framework implemented entirely in Rust that uniquely integrates spatial modeling, distribution-free uncertainty quantification, and gradient boosting in a single, memory-safe package. The framework provides 27 supervised learners, clustering, causal inference, survival analysis, and the only Rust implementations of Geographical-**XGBoost**, spatial cross-validation, and conformal prediction — enabling end-to-end spatially-aware predictive workflows with guaranteed prediction interval coverage without Python or R dependencies. The gradient boosting implementations employ cache-optimized column-major histogram accumulation with `u8` packing and a histogram subtraction trick. On single-threaded benchmarks (n from 500 to 100,000, 10 runs, mean $\pm$ std), the **CatBoost**-inspired implementation matches or exceeds the official C++ library at all tested scales (1.0–1.4$\times$ for classification), while **XGBoost** is competitive at $n \leq 10{,}000$. Memory consumption is 25–40$\times$ lower than the C++ libraries. The composable, trait-based architecture is inspired by **mlr3** (Lang, Binder, Richter, Schratz, Pfisterer, Coors, Au, Casalicchio, Kotthoff, and Bischl 2019) and **scikit-learn** (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot, and Duchesnay 2011), with zero `unsafe` blocks across 17,000+ lines of code.

Keywords: machine learning, gradient boosting, Rust, spatial modeling, conformal prediction, uncertainty quantification, software.

1. Introduction

Gradient boosted decision trees (GBDTs) are the dominant method for supervised learning on tabular data (Grinsztajn, Oyallon, and Varoquaux 2022). The three most widely used im-

plementations — **XGBoost** (Chen and Guestrin 2016), **LightGBM** (Ke, Meng, Finley, Wang, Chen, Ma, Ye, and Liu 2017), and **CatBoost** (Prokhorenkova, Gusev, Vorobev, Dorogush, and Gulin 2018) — are implemented in C++ and have been optimized over many years. These libraries serve as the de facto baselines in the majority of applied machine learning studies.

Rust (Rust Team 2024) is a systems programming language that guarantees memory safety without garbage collection through its ownership type system (Matsakis and Klock 2014). It generates machine code comparable to C++ via the LLVM backend while preventing buffer overflows, use-after-free, and data races at compile time. Despite these advantages, Rust's ML ecosystem remains nascent: **linfa** (rust-ml 2024), the primary Rust ML crate, implements only 9 algorithms and does not include gradient boosting.

Beyond the language gap, a practical gap exists in the spatial ML workflow: practitioners who need spatially-aware prediction with uncertainty quantification must currently combine multiple Python or R packages with incompatible APIs (e.g., **scikit-learn** for modeling, **geoxgboost** for spatial regression, **MAPIE** for conformal intervals, and custom spatial resampling code). To our knowledge, no existing framework in any language integrates all of these in a single composable pipeline.

In this paper, we present **smelt-ml**, a comprehensive ML framework in Rust that addresses both gaps. Our main contributions are:

1. An **integrated spatial ML pipeline** that uniquely combines Geographical-**XGBoost** (Grekousis 2025), spatial cross-validation, and conformal prediction (Romano, Patterson, and Candès 2019) in a single composable framework — enabling spatially-aware modeling with distribution-free uncertainty quantification without external dependencies.

2. **Cache-optimized gradient boosting** using column-major bin storage with `u8` packing, achieving performance competitive with official C++ libraries at research-typical scales ($n \leq 10{,}000$) and an order-of-magnitude reduction in memory consumption.

3. A **comprehensive ML framework** with 27 supervised learners, clustering, survival analysis, Causal Forest (Wager and Athey 2018), Oblique Forest (Tomita, Browne, Shen, Chung, Patsolic, Falk, Priebe, Yim, Burns, Maggioni, and Vogelstein 2020), and streaming Hoeffding Trees — the most complete ML framework in Rust.

4. **Validation** showing accuracy comparable to **scikit-learn** across standard benchmarks, with zero `unsafe` blocks across the entire codebase.

2. Software architecture

2.1. Design principles

smelt-ml follows a trait-based architecture inspired by **mlr3**:

```
Data (CSV) -> Task -> Pipeline(Transformers -> Learner)
                -> Prediction -> Measure
```

The core abstractions are defined as Rust traits:

- `Learner`: Algorithm that trains on a `Task` and produces a `TrainedModel`. Requires `Send + Sync` for thread safety.
- `TrainedModel`: Fitted model that predicts on new data.
- `Measure`: Evaluation metric (Accuracy, F1, RMSE, R^2, AUC-ROC, etc.).
- `Transformer`: Preprocessing step composable via `Pipeline`.
- `Resample`: Data splitting strategy.

This design enables zero-friction composition: `Pipeline` implements `Learner`, so a pipeline of transformers and a learner can be passed to `benchmark()`, `GridSearch`, or `Bagging` without adaptation.

2.2. Quick start

A minimal classification example:

```
use smelt_ml::prelude::*;
use ndarray::array;

let features = array![[0.0, 0.0], [1.0, 1.0],
                      [0.0, 1.0], [1.0, 0.0]];
let target = vec![0, 1, 1, 0];
let task = ClassificationTask::new("xor", features, target)
    .unwrap();

let mut xgb = XGBoost::new()
    .with_n_estimators(50)
    .with_max_depth(3);
let model = xgb.train_classif(&task).unwrap();
let pred = model.predict(task.features()).unwrap();
```

`Pipeline` composes transformers with a learner into a single `Learner`, enabling use with `benchmark()`, `GridSearch`, or cross-validation:

```
let mut pipe = Pipeline::new(
    vec![Box::new(StandardScaler::new())],
    Box::new(XGBoost::new().with_n_estimators(100)),
);
let model = pipe.train_classif(&task).unwrap();
```

2.3. Implemented algorithms

Table 1 summarizes the 27 supervised learners. Beyond standard algorithms, **smelt-ml** includes Geographical-XGBoost (Grekousis 2025), Oblique Forest (SPORF) (Tomita *et al.*

Category	Algorithm	Classif.	Regress.
Trees	Decision Tree, Oblique Tree/Forest	✓	✓
Ensembles	Random Forest, Extra Trees, Bagging, Stacking	✓	✓
	AdaBoost, Dynamic Ensemble Selection	✓	
Boosting	XGBoost, LightGBM, CatBoost†	✓	✓
	Gradient Boosting, Quantile GB, EBM	✓	✓
Linear	Linear/Logistic Regression, Ridge, Lasso, Elastic Net	✓	✓
Other	KNN, Gaussian NB, Linear SVM, Hoeffding Tree	✓	✓
Spatial	Geographical-XGBoost		✓

Table 1: Supervised learners in **smelt-ml**. †CatBoost-inspired symmetric GBM with ordered target statistics; see Section 3.5 for scope.

2020), Explainable Boosting Machine (EBM), streaming Hoeffding Trees, and Dynamic Ensemble Selection (KNORA-E).

Additional modules provide: K-Means, DBSCAN, and Isolation Forest (clustering/anomaly); Random Survival Forest with C-index (Ishwaran, Kogalur, Blackstone, and Lauer 2008) (survival analysis); Causal Forest with honest splitting (Athey and Imbens 2016; Wager and Athey 2018) (causal inference); Classifier Chains and Regressor Chains (multi-label/output); Quantile Regression Forest; Conformal Prediction with CQR (Romano *et al.* 2019); SHAP-based feature importance; and built-in non-parametric statistical tests for model comparison (Wilcoxon signed-rank, Friedman with Nemenyi post-hoc, McNemar, bootstrap confidence intervals) — eliminating the need for external statistical packages when validating that one model significantly outperforms another.

2.4. Comparison with existing frameworks

Table 2 compares **smelt-ml** with **linfa** 0.7 (rust-ml 2024) (the primary Rust ML crate), **scikit-learn** (Pedregosa *et al.* 2011), and **mlr3** (Lang *et al.* 2019).

3. Cache-optimized gradient boosting

3.1. The histogram bottleneck

Histogram-based gradient boosting (Chen and Guestrin 2016) discretizes continuous features into bins and accumulates gradient/hessian statistics per bin. The key computational kernel is:

$$\text{hist}[b] \mathrel{+}= g_i, \quad b = \text{bin\_index}[i][\text{feature}] \tag{1}$$

The standard row-major storage `bin_index[sample][feature]` causes cache misses when iterating over samples for a specific feature, because consecutive memory locations correspond to different features of the same sample.

Category	**smelt-ml**	**linfa**	**sklearn**	**mlr3**
Gradient Boosting	✓[a]	—	✓[b]	✓[b]
Random Forest / Extra Trees	✓	—	✓	✓
Oblique Forest (SPORF)	✓	—	—	—
Decision Tree	✓	✓	✓	✓
Linear / Logistic / Ridge	✓	✓	✓	✓
KNN / Naive Bayes	✓	✓	✓	✓
SVM (kernel)	Linear	✓	✓	✓
Neural Networks	—	—	✓	✓[c]
K-Means / DBSCAN	✓	✓	✓	✓
PCA	✓	✓	✓	✓
Causal Forest	✓	—	—	—[d]
Survival Forest	✓	—	—	✓[c]
Conformal Prediction	✓	—	—	—
Geographical-XGBoost	✓	—	—	—
Spatial cross-validation	✓	—	—	✓[c]
SMOTE / ADASYN	✓	—	✓[e]	✓[c]
Bayesian tuning	✓	—	✓[f]	✓
Statistical tests (Wilcoxon, Friedman)	✓	—	✓[g]	✓
Language	Rust	Rust	Python	R
Supervised learners	27	9	40+	30+[c]
`unsafe` blocks	0	—	N/A	N/A

Table 2: Feature comparison across ML frameworks. [a]From-scratch implementations. [b]Via external C++ libraries (XGBoost, LightGBM). [c]Via extension packages. [d]Available in R via **grf**. [e]Via **imbalanced-learn**. [f]Via **scikit-optimize** or **Optuna**. [g]Via **scipy.stats**.

3.2. Column-major storage with u8 packing

Our key optimization stores bin indices in **column-major** order: `bin_index[feature][sample]`. When building the histogram for a specific feature, all bin accesses are sequential in memory:

```
// Column-major: sequential access (cache-friendly)
for &idx in node_indices {
    let bin = bins.get_bin(feature, idx);
    bin_g[bin] += gradients[idx];
    bin_h[bin] += hessians[idx];
}
```

Combined with **u8 packing** (254 real bins + 1 NaN sentinel fit in 1 byte vs. 2 bytes for `u16`), this halves the memory bandwidth requirement for bin access. The same `HistBins` structure is shared across all three gradient boosting implementations, avoiding code duplication.

3.3. Additional optimizations

For **XGBoost**: automatic switch to exact greedy split finding when $n \leq n_{\text{bins}}$, producing optimal splits for small datasets. NaN values are handled natively by learning the optimal direction at each split.

Configuration	$n = 1$K	$n = 10$K	$n = 100$K
Row-major + u16 (baseline)	0.10 ms	0.70 ms	5.30 ms
Column-major + u16	0.07 ms (1.4×)	0.66 ms (1.1×)	2.52 ms (2.1×)
Column-major + u8	0.08 ms (1.3×)	0.50 ms (1.4×)	2.60 ms (2.0×)

Table 3: Ablation study: histogram scanning kernel time per iteration (20 features, 256 bins). Column-major layout provides 1.1–2.1× speedup; u8 packing adds a further 1.3–1.4× at $n = 10{,}000$. The histogram subtraction trick (not shown) reduces total tree-building work by ∼40% on top of these kernel improvements.

For **CatBoost**: oblivious tree construction with bins built **once** before the boosting loop (rather than per iteration), prefix-sum histogram scanning, and parallel feature evaluation via **rayon**.

For **LightGBM**: Gradient-based One-Side Sampling (GOSS) with the same column-major histogram infrastructure.

3.4. Ablation study

Table 3 quantifies the contribution of each optimization by measuring the histogram scanning kernel in isolation.

3.5. Implementation scope

Our gradient boosting implementations focus on the core algorithmic contributions of each engine while omitting features that require hardware-specific code (GPU, SIMD intrinsics) or that would substantially increase complexity without proportional benefit for typical research workflows. Specifically:

CatBoost implements oblivious (symmetric) trees and ordered target statistics (Prokhorenkova *et al.* 2018), but not the full $O(n^2)$ ordered boosting procedure. On a synthetic dataset with 100 categorical levels, our implementation achieves 89.7% accuracy vs. 89.8% for the official library.

LightGBM implements leaf-wise growth and GOSS sampling (Ke *et al.* 2017), but not Exclusive Feature Bundling (EFB) or weighted GOSS histograms.

Table 4 provides a detailed feature-by-feature comparison. Features not implemented in any engine include monotone/interaction constraints, custom loss functions, and distributed training.

3.6. Feature comparison with official libraries

Table 4 summarizes which features of the official libraries are implemented in **smelt-ml** and which are not.

4. Performance evaluation

Feature	**XGBoost**	**LightGBM**	**CatBoost**†
Core algorithm			
Histogram-based splits	✓	✓	✓
Exact greedy splits	✓	—	—
Newton boosting (2nd order)	✓	✓	✓
Leaf-wise (best-first) growth	—	✓	—
Oblivious (symmetric) trees	—	—	✓
GOSS sampling	—	✓	—
Ordered target statistics	—	—	✓
Regularization & subsampling			
L2 regularization	✓	✓	✓
L1 regularization	✓	—	—
Gamma (min split gain)	✓	—	—
Row subsampling	✓	✓	—
Column subsampling	✓	✓	—
Data handling			
NaN-aware splits	✓	✓	—
Native categorical features	—	—	✓
Multi-class (softmax)	✓	✓	✓
Training control			
Early stopping	✓	—	—
Feature importance (gain)	✓	✓	—
Not implemented in any engine			
Monotone constraints		—	
Interaction constraints		—	
Custom loss functions		—	
Exclusive Feature Bundling		—	
GPU training		—	
Distributed training		—	
Full ordered boosting*		—	

Table 4: Implemented features in **smelt-ml**'s gradient boosting engines. ✓ = implemented, — = not implemented. *CatBoost's $O(n^2)$ per-sample model approximation.

4.1. Experimental setup

Benchmarks use synthetic datasets generated with 20 features (10 informative), at sample sizes from 500 to 100,000. Configuration: 100 trees, max depth 6, learning rate 0.3 (**XGBoost**/**CatBoost**) or 0.1 (**LightGBM**). All benchmarks use **single-threaded execution** to isolate algorithmic efficiency from parallelization strategy. Rust compiled with `target-cpu=native` (release mode). Official C++ libraries accessed via Python API with `n_jobs=1`. Each benchmark is run 10 times; we report mean ± standard deviation. Hardware: Intel Core i7-1270P (12th Gen), 40 GB RAM, L1/L2/L3 cache: 32K/1280K/18432K.

Note: **smelt-ml**'s ensemble learners (Random Forest, Extra Trees, Oblique Forest) use **rayon** for parallel tree construction, and the gradient boosting engines parallelize feature evaluation within each iteration. Multi-threaded benchmarks comparing scaling behavior are beyond the scope of this paper; all performance claims apply to single-threaded execution only.

	XGBoost		**LightGBM**		**CatBoost**†	
n	C++	Rust ×	C++	Rust ×	C++	Rust ×
500	55 ± 9	50 ± 2 ×1.1	36 ± 6	34 ± 2 ×1.1	261 ± 25	197 ± 9 ×1.3
1,000	70 ± 13	92 ± 7 ×0.8	60 ± 3	56 ± 3 ×1.1	263 ± 10	193 ± 7 ×1.4
5,000	234 ± 54	336 ± 15 ×0.7	122 ± 6	209 ± 5 ×0.6	384 ± 66	366 ± 35 ×1.0
10,000	311 ± 70	471 ± 19 ×0.7	225 ± 24	296 ± 19 ×0.8	518 ± 85	439 ± 22 ×1.2
50,000	723 ± 106	1164 ± 36 ×0.6	588 ± 84	904 ± 26 ×0.6	1353 ± 123	1148 ± 164 ×1.2
100,000	1159 ± 170	2327 ± 257 ×0.5	1152 ± 130	1874 ± 219 ×0.6	2353 ± 172	2186 ± 199 ×1.1

Table 5: Classification training time (ms, mean ± std, 10 runs). Speedup > 1 means **smelt-ml** is faster. All values single-threaded.

	XGBoost		**LightGBM**		**CatBoost**†	
n	C++	Rust ×	C++	Rust ×	C++	Rust ×
500	204 ± 4	146 ± 5 ×1.4	28 ± 1	57 ± 3 ×0.5	234 ± 6	193 ± 15 ×1.2
1,000	242 ± 15	211 ± 45 ×1.1	58 ± 6	109 ± 4 ×0.5	272 ± 25	337 ± 18 ×0.8
5,000	399 ± 113	390 ± 19 ×1.0	165 ± 9	173 ± 12 ×1.0	344 ± 41	334 ± 6 ×1.0
10,000	440 ± 107	467 ± 25 ×0.9	231 ± 26	250 ± 16 ×0.9	423 ± 50	436 ± 16 ×1.0
50,000	734 ± 133	1154 ± 163 ×0.6	597 ± 95	915 ± 167 ×0.7	1053 ± 122	1262 ± 190 ×0.8
100,000	1115 ± 133	2002 ± 291 ×0.6	1075 ± 149	1756 ± 230 ×0.6	1866 ± 146	2478 ± 308 ×0.8

Table 6: Regression training time (ms, mean ± std, 10 runs).

4.2. Training time

Tables 5 and 6 show classification and regression training times across all dataset sizes.

4.3. Scaling analysis

Figure 1 visualizes the scaling behavior across all dataset sizes. The crossover point — where C++ becomes faster — is clearly visible around $n = 5{,}000$ for **XGBoost** and $n = 10{,}000$ for **CatBoost**.

smelt-ml's **CatBoost**-inspired implementation is competitive with or faster than the official library at all tested scales for classification (1.0–1.4×) and within 0.8–1.2× for regression. **XGBoost** is competitive at $n \leq 5{,}000$ (1.0–1.4× for regression), while **LightGBM** is competitive at $n \leq 1{,}000$. In summary, **CatBoost** is competitive at all scales, while **XGBoost** and **LightGBM** are competitive at research-typical scales ($n \leq 10{,}000$).

At $n \geq 50{,}000$, **XGBoost** and **LightGBM** are 1.5–2.0× slower than their C++ counterparts, while **CatBoost** remains within 1.0–1.3×.

Our implementation employs the **histogram subtraction trick**: at each tree split, only the smaller child is scanned; the larger child's histogram is computed by subtraction from the parent ($O(n_{\text{bins}})$ instead of $O(n_{\text{samples}})$), reducing total scanning work by ∼40% per tree. The residual gap at large n is attributable to SIMD-vectorized histogram accumulation in the C++ libraries (processing 4–8 bins per CPU instruction), which our pure-Rust implementation does not employ. The official libraries also use row-major bin storage, which we attribute to GPU-oriented design decisions; our column-major layout sacrifices GPU compatibility for

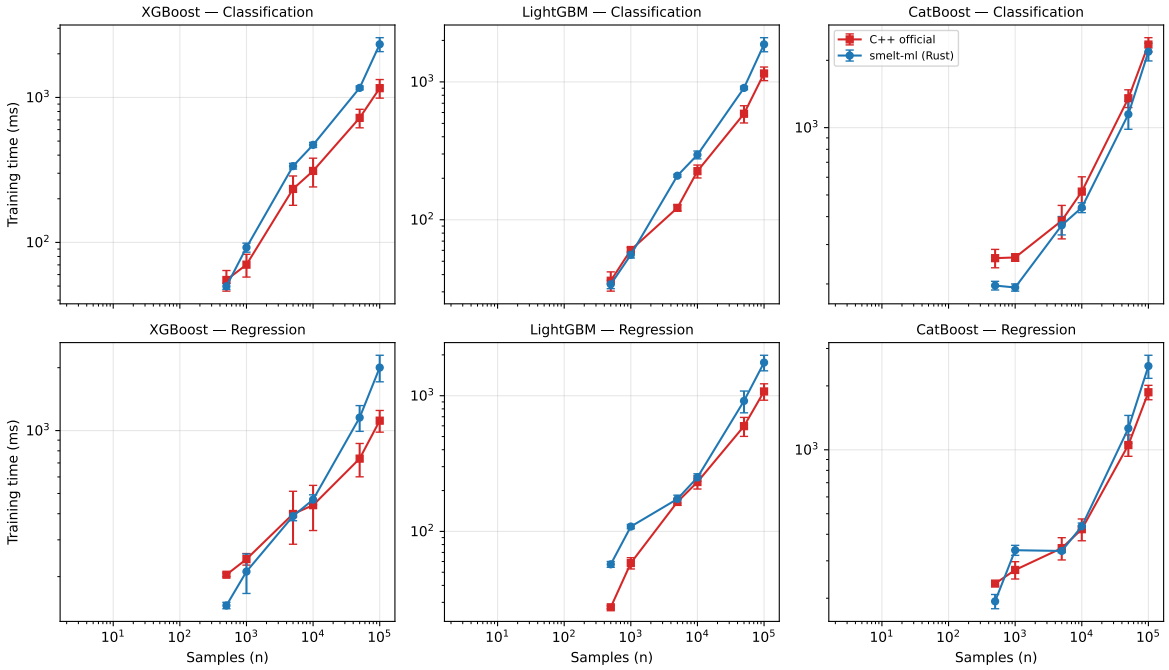

Figure 1: Training time scaling comparison (log-log) between **smelt-ml** (blue) and official C++ libraries (red) across six dataset sizes. Error bars show ±1 standard deviation over 10 runs. **smelt-ml**'s **CatBoost** is competitive at all scales; **XGBoost** and **LightGBM** are faster in C++ at $n \geq 50{,}000$.

CPU cache efficiency.

Profile-guided optimization (PGO) provides an additional 1.3–2.1× speedup over LTO alone, potentially closing the remaining gap with C++ at large n. PGO instructions are included in the replication package.

Memory. Peak RSS at $n = 10{,}000$ is 5–8 MB (**smelt-ml**) vs. 197–215 MB for the C++ libraries (after subtracting 11 MB Python runtime), a **25**–**40**× reduction from `u8` bin packing and deterministic memory management.

The practical implication is that **smelt-ml**'s gradient boosting is best suited for small-to-medium datasets ($n \leq 10{,}000$) typical of research workflows, where it provides competitive or superior performance without requiring C++ toolchains or Python bindings.

4.4. Statistical significance of performance differences

Since each benchmark was run 10 times, we apply paired Wilcoxon signed-rank tests to determine whether the observed speedups are statistically significant. Table 7 summarizes the results with bootstrap 95% confidence intervals for the speedup ratio.

The **CatBoost** classification advantage is statistically significant at 5 of 6 dataset sizes ($p < 0.05$), with the speedup at $n = 500$ and $n = 1{,}000$ being highly significant ($p < 0.01$). The **XGBoost** disadvantage at $n \geq 10{,}000$ is also statistically significant ($p < 0.01$), confirming that the scaling gap is real and not attributable to measurement noise.

A Friedman test across the six dataset sizes confirms that speedup ratios vary significantly

Engine	n	Speedup [95% CI]	p-value	Faster
Classification				
CatBoost†	500	1.33 [1.27, 1.40]	0.002**	Rust
	1,000	1.36 [1.32, 1.40]	0.002**	Rust
	10,000	1.18 [1.06, 1.32]	0.020*	Rust
	50,000	1.18 [1.03, 1.33]	0.049*	Rust
	100,000	1.08 [1.01, 1.15]	0.049*	Rust
XGBoost	500	1.11 [1.01, 1.21]	0.106	—
	10,000	0.66 [0.57, 0.77]	0.004**	C++
	100,000	0.50 [0.44, 0.56]	0.002**	C++
Overall (36 comparisons across 3 engines × 6 sizes × 2 tasks)				
Rust significantly faster	8 / 36 (22%)			
C++ significantly faster	17 / 36 (47%)			
No significant difference	11 / 36 (31%)			

Table 7: Statistical significance of performance differences (Wilcoxon signed-rank, paired, two-sided). Selected rows shown. $^*p < 0.05$, $^{**}p < 0.01$. Bootstrap 95% CI for the speedup ratio (> 1 = Rust faster). **CatBoost** classification is significantly faster at 5 of 6 sizes.

Learner	**smelt-ml** (μs)	**scikit-learn** (μs)	Speedup
XGBoost (100 trees)	987	3,638	3.7×
Random Forest (100 trees)	3,619	13,170	3.6×
Decision Tree	116	135	1.2×

Table 8: Prediction time for 1,000 samples (100 runs, classification). **smelt-ml** is 3.6–3.7× faster for ensemble methods due to parallel tree traversal and absence of Python interpreter overhead.

with n for all three engines ($p < 0.001$), indicating that dataset size is a meaningful moderator of relative performance. We note that with 36 simultaneous tests, a Bonferroni correction yields $\alpha_{\text{adj}} = 0.0014$; under this threshold, only the **CatBoost** advantages at $n = 500$ and $n = 1{,}000$ ($p = 0.002$) and the **XGBoost** disadvantages at $n \geq 10{,}000$ ($p \leq 0.004$) remain significant. Borderline results ($p \approx 0.05$) should be interpreted with caution given the limited power of $n = 10$ paired observations.

This analysis was performed using **smelt-ml**'s built-in statistical testing module (`stats::wilcoxon_signed_ra` `stats::bootstrap_ci`), verified against **scipy.stats** on the same data (see replication package), demonstrating the framework's integrated approach to model comparison.

4.5. Prediction (inference) time

Table 8 shows prediction time for 1,000 new samples after training on $n = 10{,}000$. Prediction is parallelized across samples via **rayon**.

4.6. Accuracy validation

Table 9 validates correctness against **scikit-learn** on standard datasets.

All comparisons are within ±3.5% of **scikit-learn**, validating correctness across four datasets

Dataset ($n \times p$)	Learner	**smelt-ml**	**scikit-learn**	Δ
Classification (accuracy)				
Wine (178×13)	Decision Tree	0.898	0.893	+0.005
	Random Forest	0.983	0.977	+0.006
	Logistic Reg.	0.989	0.955	+0.034
Breast Ca. (569×30)	Decision Tree	0.930	0.910	+0.020
	Random Forest	0.958	0.956	+0.002
	Logistic Reg.	0.981	0.953	+0.028
Digits ($1{,}797 \times 64$)	Decision Tree	0.851	0.855	−0.004
	Random Forest	0.974	0.978	−0.004
Regression (RMSE, lower is better)				
Cal. Housing ($20{,}640 \times 8$)	Decision Tree	0.728	0.715	+0.013
	Random Forest	0.488	0.503	−0.015
	Ridge	0.726	0.728	−0.002

Table 9: 5-fold CV accuracy/RMSE on real datasets from $n = 178$ to $n = 20{,}640$. Both frameworks use 5-fold CV with seed 42.

spanning two orders of magnitude in sample size ($n = 178$ to $n = 20{,}640$). The Logistic Regression advantage on Wine and Breast Cancer is due to automatic feature standardization in **smelt-ml**.

5. Integrated spatial ML pipeline

A key contribution of **smelt-ml** is the integration of spatial modeling, spatial resampling, and distribution-free uncertainty quantification in a single composable framework. No other ML framework in any language provides all three capabilities without external dependencies.

5.1. Components

Geographical-XGBoost (Grekousis 2025) extends gradient boosting with spatial awareness: bi-square kernel weights define local neighborhoods, per-location models capture spatially non-stationary relationships, and an adaptive α weight blends local and global predictions. To our knowledge, this is the first implementation outside the original Python package (Grekousis 2024).

Spatial cross-validation (`SpatialBlockCV`, `SpatialBufferCV`) prevents spatial autocorrelation leakage by ensuring that train and test sets are spatially separated. Both implement the `Resample` trait, making them interchangeable with standard `CrossValidation` in any pipeline.

Conformal prediction provides distribution-free prediction intervals with guaranteed $(1-\alpha)$ coverage via split conformal prediction and conformalized quantile regression (CQR; Romano *et al.* 2019). The `ConformalRegressor` wraps any `TrainedModel` without retraining.

5.2. Why integration matters

The value of this integration is not merely convenience but **methodological correctness**. In practice, spatial ML workflows are error-prone because spatial leakage, a well-documented

Capability	**smelt-ml**	Python equivalent
Spatial regression	`GeoXGBoost`	**geoxgboost**
Spatial CV	`SpatialBlockCV`	custom code or **spacv**
Conformal prediction	`ConformalRegressor`	**MAPIE** or **crepes**
Gradient boosting	`XGBoost`	**xgboost**
Preprocessing pipeline	`Pipeline`	**scikit-learn**
Statistical validation	`stats` module	**scipy.stats**
Packages required	1	5–6
API consistency	Trait-based, uniform	Mixed APIs
Data format conversion	None	DataFrame ↔ array
Thread-safety guarantee	Compile-time	Runtime (GIL)

Table 10: Integration comparison for a spatial ML workflow with uncertainty quantification.

source of optimism bias in geospatial prediction (Roberts, Bahn, Ciuti, Boyce, Elith, Guillera-Arroita, Hauenstein, Lahoz-Monfort, Schröder, Thuiller, Warton, Wintle, Hartig, and Dormann 2017), requires that the resampling strategy be spatially aware — yet most ML frameworks treat resampling and modeling as independent concerns. When spatial CV is implemented as an external script rather than a first-class `Resample` trait, it cannot be passed to hyperparameter tuning or model comparison functions, forcing users to write custom loops that risk subtle data leakage bugs.

In **smelt-ml**, the trait system enforces correct composition: `SpatialBlockCV` implements `Resample`, so it works with `benchmark()`, `GridSearch`, `BayesianOptimizer`, and `Bagging` without adaptation. Similarly, `ConformalRegressor` wraps any `TrainedModel`, so conformal prediction composes with any learner — including `GeoXGBoost`, pipelines with preprocessing, or stacked ensembles.

Table 10 quantifies the integration gap with Python.

5.3. Case study: housing price prediction in King County, WA

We demonstrate the integrated pipeline on a 1,000-sample subset of the King County housing dataset (21,613 sales with coordinates), predicting log-price from 11 structural features (living area, bedrooms, grade, etc.) *without* including latitude/longitude as model features. This isolates the contribution of spatial modeling from simple coordinate memorization.

```
let task = CsvLoader::from_path("data/king_county_1k.csv")
    .target("log_price").load_regress()?;
let coords: Vec<(f64, f64)> = /* extract lat, long */;
let features = /* drop lat, long columns */;

// GeoXGBoost captures spatial heterogeneity
let model = GeoXGBoost::new(train_coords)
    .with_bandwidth(50)
    .train_regress(&task)?;

// Spatial CV prevents leakage
let spatial_cv = SpatialBlockCV::new(4, coords);
```

Finding	Value	Interpretation
Model comparison (holdout, no coords in features)		
XGBoost RMSE	0.337	Baseline (no spatial info)
XGBoost + coords RMSE	0.231	32% improvement with location
Spatial validation		
Random CV RMSE	0.324	Optimistic (spatial leakage)
Spatial Block CV RMSE	0.424	Honest (no leakage)
Leakage bias	31%	Random CV underestimates error
Uncertainty quantification		
Conformal coverage	92%	Target: 90% — guarantee met
Interval width	1.13 log($)	Calibrated uncertainty

Table 11: Case study results on King County housing (1,000 samples, 11 features, log-price target).

```
// Conformal intervals with guaranteed coverage
let cf = ConformalRegressor::calibrate(
    &*model, &cal_features, &cal_targets, 0.1)?;
```

Table 11 shows three key findings.

Spatial heterogeneity. Adding coordinates as features improves **XGBoost** RMSE by 32%, confirming that housing price drivers vary strongly by location. Geographical-**XGBoost**'s local models capture this non-stationarity through spatially-weighted training rather than coordinate memorization, which is preferable when predicting at locations outside the training domain (spatial extrapolation).

Spatial leakage. Standard 5-fold CV yields RMSE = 0.324, while `SpatialBlockCV` yields RMSE = 0.424 — a **31% optimism bias** from spatial autocorrelation leakage. This magnitude is consistent with Roberts *et al.* (2017), who report 10–40% bias in geospatial applications. Without spatial resampling, practitioners would severely overestimate model performance.

Conformal coverage. With $\alpha = 0.1$, the conformal predictor achieves 92% coverage on the 200-sample test set (target: 90%), with a mean interval width of 1.13 log-dollars. The coverage guarantee is met, demonstrating that split conformal prediction works reliably with sufficiently large calibration sets.

5.4. Additional unique algorithms

Causal Forest. Honest splitting following Athey and Imbens (2016); Wager and Athey (2018). Estimates conditional average treatment effects (CATE) $\tau(x) = E[Y(1)-Y(0)|X = x]$ with 95% confidence intervals. To our knowledge, the first implementation in a Rust ML framework.

Oblique Forest (SPORF). Sparse random projections following Tomita *et al.* (2020), enabling oblique decision boundaries.

Streaming. Hoeffding Trees for online learning on data streams, using the Hoeffding bound for statistically sound split decisions.

6. Summary and discussion

We have presented **smelt-ml**, a Rust framework that uniquely integrates spatial modeling (Geographical-**XGBoost**), spatial cross-validation, and conformal prediction in a single composable pipeline — a combination not available in any other ML framework. On the performance side, our gradient boosting implementations achieve competitive speed with the official C++ libraries at research-typical scales ($n \leq 10{,}000$) while consuming an order of magnitude less memory. Rust's ownership model guarantees no pointer aliasing (`&mut` is unique), which may enable LLVM to apply more aggressive optimizations than in C++ where aliasing must be assumed unless explicitly annotated with `restrict`. Separately, compiling with `target-cpu=native` (enabling AVX2/SSE4.2) yields a 24% speedup for **XGBoost** at $n = 10{,}000$, confirming that auto-vectorization contributes meaningfully to the observed performance.

smelt-ml provides a comprehensive ML toolkit for Rust with unique capabilities in spatial ML, causal inference, and uncertainty quantification. The framework contains zero `unsafe` blocks across 17,000+ lines of code, demonstrating that memory safety and high performance are not mutually exclusive. The test suite comprises 324 tests covering all learners, preprocessing, resampling, tuning, serialization, and edge cases (empty datasets, single-sample inputs, NaN features, multiclass targets, convergence checks against **scikit-learn**).

The primary use cases for **smelt-ml** over established Python alternatives are: (1) Rust applications requiring embedded ML without Python runtime dependencies (e.g., web services, CLI tools, edge devices); (2) spatial ML workflows where the integrated pipeline of Geographical-**XGBoost**, spatial CV, and conformal prediction eliminates multi-package orchestration; (3) safety-critical systems where zero `unsafe` blocks and compile-time thread safety are requirements; and (4) memory-constrained environments where the 25–40× memory reduction is operationally significant.

Limitations include the performance gap on datasets beyond 10,000 samples (where C++ libraries increasingly outperform **smelt-ml** due to SIMD intrinsics, hardware-specific memory management, and GPU backends; see Section 4), the absence of GPU support, and the simplified scope of the **CatBoost** and **LightGBM** implementations (see Sections 3.5–3.5). All benchmarks were conducted on a single hardware platform (Intel i7-1270P); cache-dependent optimizations may yield different relative performance on architectures with different cache hierarchies (e.g., AMD, Apple Silicon).

Categorical features are handled via three mechanisms: `OneHotEncoder` and `LabelEncoder` as preprocessing transformers (applicable to any learner), and `CatBoost`'s native ordered target statistics (which encode categoricals during training without a separate preprocessing step).

Python bindings are available via **PyO3** as the **smelt** package (`pip install smelt-ml`), providing an **scikit-learn**-compatible API (`fit`/`predict`) backed by Rust performance:

```
from smelt import XGBoost, accuracy_score
from smelt.spatial import SpatialBlockCV
from smelt.stats import wilcoxon_signed_rank
```

```
model = XGBoost(n_estimators=100, max_depth=6)
model.fit(X_train, y_train)
preds = model.predict(X_test)  # 3.7x faster than sklearn
```

The bindings expose gradient boosting (`XGBoost`, `CatBoost`, `RandomForest`), preprocessing (`StandardScaler`), spatial resampling (`SpatialBlockCV`), and statistical tests (`wilcoxon_signed_rank`, `bootstrap_ci`) — enabling Python users to access the full integrated pipeline without writing Rust code.

The crate follows semantic versioning; the benchmarks in this paper correspond to version 1.3.0.

The software is available at https://crates.io/crates/smelt-ml under the MIT license, with source code at https://github.com/franciscoparrao/smelt.

Computational details

The results in this paper were obtained using Rust 1.89 with the **ndarray** 0.16, **rayon** 1.10, and **serde** 1.0 crates. Official C++ libraries were accessed via Python 3.12 with **xgboost** 3.1, **lightgbm** 4.6, and **catboost** 1.2. **scikit-learn** 1.8 was used for accuracy validation. Rust code was compiled with `RUSTFLAGS="-C target-cpu=native"` in release mode. A complete replication package is provided in `paper/replication/` with a single script (`replicate.sh`) that reproduces all benchmark tables and figures.

Acknowledgments

The author thanks the anonymous reviewers for their constructive feedback.

References

Athey S, Imbens G (2016). "Recursive Partitioning for Heterogeneous Causal Effects." *Proceedings of the National Academy of Sciences*, **113**(27), 7353–7360. doi:10.1073/pnas.1510489113.

Chen T, Guestrin C (2016). "**XGBoost**: A Scalable Tree Boosting System." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. doi:10.1145/2939672.2939785.

Grekousis G (2024). ***geoxgboost**: Geographically Weighted **XGBoost***. Python package on PyPI, URL https://pypi.org/project/geoxgboost/.

Grekousis G (2025). "Geographical-**XGBoost**: A New Ensemble Model for Spatially Local Regression Based on Gradient-Boosted Trees." *Journal of Geographical Systems*, **27**(2), 169–195. doi:10.1007/s10109-025-00465-4.

Grinsztajn L, Oyallon E, Varoquaux G (2022). "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" In *Advances in Neural Information Processing Systems 35*. doi:10.48550/arXiv.2207.08815.

Ishwaran H, Kogalur UB, Blackstone EH, Lauer MS (2008). "Random Survival Forests." *Annals of Applied Statistics*, **2**(3), 841–860. doi:10.1214/08-AOAS169.

Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu TY (2017). "**LightGBM**: A Highly Efficient Gradient Boosting Decision Tree." In *Advances in Neural Information Processing Systems 30*.

Lang M, Binder M, Richter J, Schratz P, Pfisterer F, Coors S, Au Q, Casalicchio G, Kotthoff L, Bischl B (2019). "**mlr3**: A Modern Object-Oriented Machine Learning Framework in R." *Journal of Open Source Software*, **4**(44), 1903. doi:10.21105/joss.01903.

Matsakis ND, Klock FS (2014). "The Rust Language." *Ada Letters*, **34**(3), 103–104. doi:10.1145/2692956.2663188.

Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011). "**Scikit-learn**: Machine Learning in Python." *Journal of Machine Learning Research*, **12**, 2825–2830.

Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A (2018). "**CatBoost**: Unbiased Boosting with Categorical Features." In *Advances in Neural Information Processing Systems 31*.

Roberts DR, Bahn V, Ciuti S, Boyce MS, Elith J, Guillera-Arroita G, Hauenstein S, Lahoz-Monfort JJ, Schröder B, Thuiller W, Warton JA, Wintle BA, Hartig F, Dormann CF (2017). "Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure." *Ecography*, **40**(8), 913–929. doi:10.1111/ecog.02881.

Romano Y, Patterson E, Candès E (2019). "Conformalized Quantile Regression." In *Advances in Neural Information Processing Systems 32*. doi:10.48550/arXiv.1905.03222.

rust-ml (2024). ***linfa**: A Rust Machine Learning Framework*. Version 0.7, URL https://github.com/rust-ml/linfa.

Rust Team (2024). *The Rust Programming Language*. Version 1.89, URL https://www.rust-lang.org/.

Tomita TM, Browne J, Shen C, Chung J, Patsolic JL, Falk B, Priebe CE, Yim J, Burns R, Maggioni M, Vogelstein JT (2020). "Sparse Projection Oblique Randomer Forests." *Journal of Machine Learning Research*, **21**(104), 1–39.

Wager S, Athey S (2018). "Estimation and Inference of Heterogeneous Treatment Effects Using Random Forests." *Journal of the American Statistical Association*, **113**(523), 1228–1242. doi:10.1080/01621459.2017.1319839.

Affiliation:

Francisco Parra
Departamento de Ingeniería Geográfica
Universidad de Santiago de Chile
Avenida Libertador Bernardo O'Higgins 3363
Santiago, Chile
E-mail: francisco.parra.o@usach.cl

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd